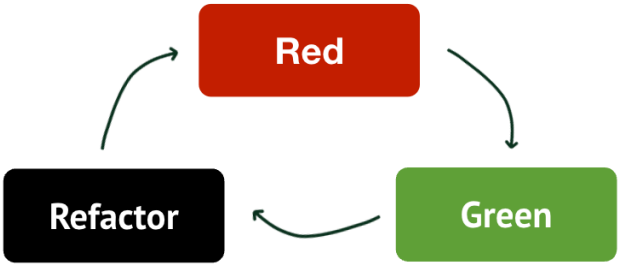
Emily Bache's Guide to Code Katas for learning TDD

Test Driven Development (TDD) is an approach to software development that puts tests and testability at the heart of the design process. Code Katas are small, fun exercises that can help you to learn TDD. This document aims to help you get started with both.



The TDD Cycle - Overview

This is a summary of "Canon TDD" described by Kent Beck in more detail his article:

- **Test List**: Do just enough analysis to work out how to approach the problem and define the first test case. Note down your ideas in the form of a test list.
- **Red**: Write a test for the functionality you wish to implement. This test will initially fail, as the functionality is not yet present. Run the test to see it fail. This confirms that the test is detecting the missing functionality and gives a good error message.
- **Green**: Write the minimum amount of code necessary to make the test pass. Verify that the test now passes with the new code.
- **Refactor**: Clean up the code while ensuring that all tests still pass.

This process for writing code takes some skill and discipline, and most programmers find it difficult to get started with it in their production code. Those that have mastered it often report they are much more effective and would not like to work in any other way. Code Katas can help you to get started learning TDD.

Code Katas for learning TDD

A Code Kata is a small programming exercise you can use to improve your skills and habits. Usually they are a fun distracting little problem in a domain that's familiar enough to be both interesting and not too hard to grasp. Coding as a process involves hundreds of minute-by-minute decisions about how to spend your time and attention, and habits are your default decisions. Training on code katas can help you to adjust your habits in favour of taking smaller steps and running tests more often. You can practice breaking down problems into smaller pieces and building solutions test-by-test with TDD.

If you work on a coding problem that is too difficult, it will take all of your brainpower and you won't have enough left over to reflect on the *way* you are working. To learn TDD you need to consciously reduce the size of your steps and increase the frequency of running tests. That is why code katas are often quite small problems that are not all that difficult for professional developers to solve. The challenge comes from the way you approach them. It's similar to how musicians play scales and etudes to prepare for performing more complex compositions, or martial artists practice kata exercises to prepare for sparring. That's of course where the name comes from.

Practice makes perfect

Habits to a large extent come from repetition. There are nearly no downsides to doing the same code kata more than once. The second and third times you solve it, you already know how a solution could look, so more of your brainpower is available for the *way* you solve it. You can think more about the order of tests, identify even smaller steps you could take, and experiment with refactoring more earlier or later in the process. When you've solved a kata ten or more times, it becomes all about fluency and 'muscle memory' of the steps to get good tests and a good design.

A first Code Kata

If you've never done a code kata before, I suggest beginning with Leap Years. This problem can be solved with a pure function containing a few lines of logic. The important thing is not to simply write the entire function at once. In TDD you break a problem down into test cases that you can implement one at a time. Remember, you're primarily training coding habits - you already know how to solve a mathematical problem like this with code. Focus on taking small steps, turning an example into a test and writing only enough code to make it pass.

Katas for pure functions

The easiest kind of code to build with TDD is pure functions - they are inherently testable because the test only needs to control the arguments and check the return value. It's worth getting very good at building this kind of code with TDD. Pure functions are the basis for building cohesive objects and modules. I suggest you continue with katas that practice this skill, for slightly more complex functions. For example Tennis, Christmas Song and Shopping Basket.

Katas where Modules and Classes come in

My preference is for object oriented design but actually you can solve all these Katas in a more functional style too. The difference when you get to katas like Mars Rover, Yatzy and Monty Hall is that there is enough logic that you need to organize the code into classes or modules - there will be several functions or methods involved in a solution. It also gets more challenging to break the problem into a test list you can implement one by one and keep all previous tests passing.

Side Effects and Mocks

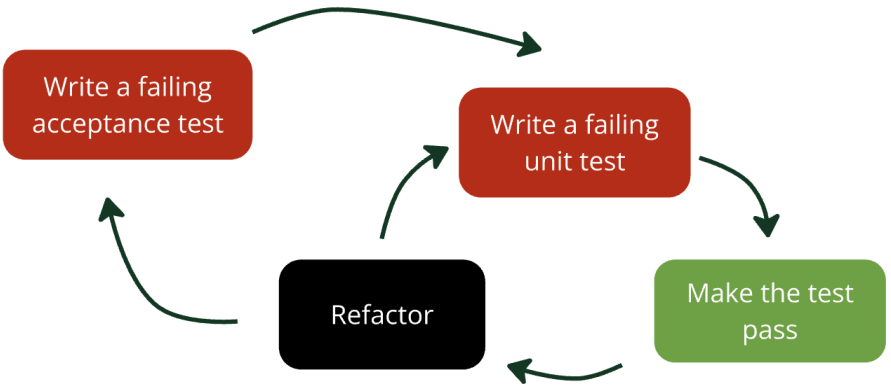
According to Michael Feathers' definition from his book "Working Effectively with Legacy Code", a test is not a *unit* test if:

- It talks to a database
- It communicates across the network
- It touches the file system
- You have to do things to your environment to run it (eg, change config files & environment variables)

In TDD the preference is to use *unit* tests to drive the design, since they are very fast to run and give good feedback on whether your logic is working correctly. All the katas I've mentioned so far can largely be completed with straightforward unit tests. However, code that solves real world problems will need to interact with databases and other systems over the network. Often we use Test Doubles to replace those things so we can keep writing fast unit tests. There are some katas specifically designed to practice this skill, for example Birthday Greetings.

Double-loop TDD

In TDD the tests have two roles, firstly to verify the lowest level of logic does what we, the programmer, think it should do. Secondly, to express the intent and needs of the user of the software. Ideally all our tests are written in a way that someone familiar with the problem domain can read and understand what is being tested. In practice, you often need two kinds of tests. To complement unit tests, we also use acceptance tests in a Double-loop TDD process:



Double-loop TDD is described in Steve Freeman and Nat Pryce's book "Growing Object Oriented Software, Guided by Tests". Most of the book comprises a full worked example to demonstrate what they mean. That example is significantly larger than the kinds of code katas you usually use to practice the habits of TDD. Unfortunately, I think you need a project of that kind of scale before it makes sense to split your tests into "unit" and "acceptance" levels. In most code katas the two roles can be combined in the same test cases - unit tests that check the logic and also use language from the domain in the test name and implementation.

In order to learn double-loop TDD you need to move onto larger scale problems. I recommend you start your own pet project in a technology of your choice. Find an opportunity to code where there are no deadlines except those you set for yourself. You can focus on not only solving a problem you care about, and also spend time and energy reflecting on *how* you are building it. This is no longer a classic Code Kata but it should still be fun to work on and help you learn skills and habits you can apply in your normal production code work.

Next Steps

If you'd like to learn more about TDD, do take a look at this list of Learning Hours - sessions you can do with your team to improve your coding habits and skills with Test Driven Development. Don't forget to sign up for the Samman Society Newsletter to hear about new resources and other interesting opportunities. You get better at TDD by training your habits, and that comes from practice. Code Katas are fun little problems that you can practice on. Happy Coding!